Head-to-head Evaluation of Translation-First vs Agentic Generate→Validate→Repair Pipelines for Intent-Driven NetOps Changes — Validator-Acceptance Parity under Shared-Candidate Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered, artifact-archived paired experiment (CAND-2026-001-prereg-v1) that compared two operational architectures for intent→configuration NetOps changes across heterogeneous vendor CLIs and Mininet-derived topologies: (A) translation-first (constrained vendor DSL → formal verifier → solver) and (B) agentic generate→validate→repair (hosted LLM + deterministic validators). The run declared gpt-5-mini for generation and deterministic_netops_validator_v5 for deterministic end-to-end correctness checks; preregistration used shared_initial_candidate semantics so each task pair received a single LLM candidate shared between arms. Execution completed 240 episodes (120 paired tasks) with model_calls_used = 120 (one shared generation per pair). Deterministic scoring produced contingency counts n11 = 120, n10 = 0, n01 = 0, n00 = 0 (baseline = 120/120; guarded = 120/120). Paired difference (A - B) = 0.00; paired-bootstrap 95% CI = [0.00, 0.00] (10,000 resamples, seed 1729); exact McNemar two-sided p = 1.0. Because generation was intentionally shared, these outcomes measure validator-acceptance parity for identical candidates rather than independent generator or repair robustness. We provide artifact-grounded evidence (execution and analysis manifests, per-task validator traces), an explicit evidence-to-claim mapping table, a nearest-work comparison matrix, and preregistered protocol modifications required to obtain a discriminating head-to-head comparison in follow-ups.**

I. Introduction

Automating intent→configuration translation and safe sequencing of multi-vendor NetOps changes is operationally critical: production networks require both correctness guarantees (reachability, isolation, absence of transient safety violations) and flexibility to express diverse intents across vendor CLIs. Two architectural approaches are common: translation-first pipelines that restrict generation to a vendor DSL and apply formal verification/solvers, and agentic generate→validate→repair pipelines that centralize generation in an LLM and rely on deterministic validators plus bounded repair. Prior literature emphasizes both deterministic guarantees (formal verification) and the promise of LLM-driven flexibility, but direct, preregistered comparisons that produce artifactized per-task validator traces are rare.

This study's preregistered head-to-head question was: for intent-driven network changes across heterogeneous vendor CLIs and realistic emulator topologies, which architecture yields higher end-to-end operational correctness (measured by deterministic validation: reachability and policy isolation) — (A) translation-first or (B) agentic generate→validate→repair? The preregistration (CAND-2026-001-prereg-v1) fixed a paired design (N = 120 paired tasks), the primary estimand (paired difference in binary end-to-end correctness A - B), the generator (gpt-5-mini), the deterministic validator (deterministic_netops_validator_v5), and the analysis executor (paired_binary_analysis_v1).

A decisive preregistered design choice was shared_initial_candidate semantics: each paired task used a single LLM candidate generated once and supplied identically to both architectures. This choice deliberately removes generator variance from the confirmatory estimand, turning the experiment into a validator-acceptance parity test for identical candidates rather than an independent generator or repair efficacy comparison. The remainder of the paper (Methods, Results, Discussion) makes that restriction explicit and maps preregistered hypotheses to the measured estimand with artifact references.

II. Related Work

Two research traditions frame our contribution. Formal translation→verification systems provide soundness and solver-assisted synthesis for network configurations; they emphasize deterministic guarantees and incremental verification [10,11,12]. LLM-driven NetOps and agentic AIOps work investigates flexible generation and iterative repair, and stresses the role of deterministic validators to bound hallucinations and unsafe proposals [3,4,5]. Representative nearest works include INTA (ICNP 2025) that evaluates intent translation strategies and LLM agents [3], Mahmood & Song (IFIP Networking 2026) showing self-correcting multi-agent verifier pipelines [4], and Weighted NetKAT/formal verification literature that underpins translation-first guarantees [5,10–12].

Nearest-work comparison (compact). Table 1 (in manuscript) summarizes four experimental axes—independent generation sampling, repair-loop instrumentation, validator acceptance focus, and emulator/hardware realism—across selected prior work and this run. The present run uniquely combines preregistration, archived paired traces, and deterministic per-task validator artifacts under shared_initial_candidate semantics. Unlike prior independent-generation experiments, this run isolates downstream orchestration and validation behavior given identical candidates.

III. Methodology

Preregistration and execution contract. The preregistration (CAND-2026-001-prereg-v1) fixed the primary estimand (paired difference in binary end-to-end correctness A - B), N_confirmatory = 120 paired tasks, generator = gpt-5-mini, validator = deterministic_netops_validator_v5 (v5.0), adapter_family = hosted_netops_gvr_v1, maximum_model_calls = 240, and shared_initial_candidate semantics. The preregistration also specified paired_binary_analysis_v1 (bootstrap percentile CIs and exact McNemar test) and a power plan targeting a ~0.15 paired difference with N=120. The preregistration artifact is archived and its checksum is recorded in the execution manifest.

Task generation and difficulty calibration. Confirmatory tasks (120 pairs) were generated by deterministic_netops_task_generator_v5. Each manifest entry (execution/task_manifest.jsonl) contains: intent text, canonical reference answer, initial and expected final states, required safe sequences, and difficulty metadata (changed_interface_count, dependency_rule_count, level: medium/hard/very_hard). Representative entries (e.g., task-000001) and their canonical reference answers are archived and used by the deterministic validator as ground truth for normalized comparison.

Generation and shared-candidate semantics. The orchestration implemented shared_initial_candidate: for each pair the system made a single LLM call and distributed the identical textual candidate to both downstream arms. Execution accounting (execution/execution_manifest.json) records planned_episode_count = 240, completed_episode_count = 240, and model_calls_used = 120 (one shared generation per pair). Provider audit (analysis/deterministic_reconciliation.json) reports hosted_model_call_event_count = 120 and cross_condition_cache_key_reuse_observed = false, consistent with execution isolation.

Validator rules and scoring. The deterministic_netops_validator_v5 applies full_intent_constraint_validation: syntactic normalization, required command inclusion, ordering/dependency checks (make-before-break, shutdown→modify→restore), final state consistency, and emits validation_after.valid (boolean). A configuration is scored success (1) only when validation_after.valid == true. Per-task scoring JSONs (execution/scoring/task-000XXX-*.json) record normalized_response, parsed_command_count, required_command_count, validation_before/after snapshots, repair_applied flags and the scorer id/version. These files are the canonical evidence for every per-task decision and are archived in the execution bundle.

Analysis plan and exact procedures. Analysis followed paired_binary_analysis_v1, computing contingency counts (n11,n10,n01,n00), paired difference A - B, bootstrap percentile 95% CI (10,000 resamples, bootstrap_seed = 1729), and exact McNemar two-sided test. Analysis artifacts include analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl (bootstrap parameters), and analysis/deterministic_reconciliation.json. Secondary estimands (repair coverage, mean repair iterations) were preregistered; their measurement depends on nonzero repair_applied flags in scoring JSONs.

IV. Results

Execution accounting and primary numeric outcomes. The execution manifest reports planned_episode_count = 240, completed_episode_count = 240, failed_episode_count = 0 and model_calls_used = 120 (one shared generation per paired task). Provider audit and deterministic reconciliation confirm hosted_model_call_event_count = 120, cache_miss_count = 120, cross_condition_cache_key_reuse_observed = false, and complete_pair_count = 120, consistent with the preregistered shared_initial_candidate execution semantics.

Deterministic scoring and contingency counts. The complete paired contingency is n11 = 120, n10 = 0, n01 = 0, n00 = 0. Marginal success rates are baseline = 120/120 = 1.0 and guarded = 120/120 = 1.0 (analysis/condition_summary.csv records these counts explicitly). The paired difference (A - B) equals 0.00. The paired bootstrap percentile 95% CI computed with 10,000 resamples and bootstrap_seed = 1729 is [0.00, 0.00]; the exact McNemar two-sided p-value is 1.0. These numbers are direct outputs of the preregistered paired_binary_analysis_v1 executor and are archived in analysis/paired_contingency_table.csv and analysis/analysis_log.jsonl.

Repair instrumentation and secondary estimands. Across all 240 episodes the archived per-task scoring JSONs show repair_applied = false (aggregated repair_applied count = 0). Consequently, preregistered secondary estimands concerning repair coverage and mean repair iterations are unobserved in this execution. The absence of repair events is reflected uniformly in execution/scoring/*.json where validator_trace.repair_applied = false and repair_provider_trace_path is null.

Representative per-task diagnostics. Representative artifacts illustrate the validator acceptance logic and task difficulty span. Example highlights from archived scoring JSONs:

• pair-000001 (task-000001): difficulty.level = "medium"; parsed_command_count = 4, required_command_count = 4. The recorded normalized_reference_answer differs in command ordering from the normalized_response (vlan and mtu lines swapped), yet validation_after.valid = true — demonstrating that the validator's normalization and dependency checks accept ordering variants when required commands and final state constraints are satisfied. • pair-000002 (task-000002): difficulty.level = "hard"; parsed_command_count = 2, required_command_count = 2; normalized_response matches the reference exactly and validation_after.valid = true. • pair-000003 (task-000003): difficulty.level = "hard"; parsed_command_count = 6, required_command_count = 6; again validation_after.valid = true. These representative traces show tasks across medium→hard levels, with parsed_command_count and required_command_count recorded per task and used by the deterministic validator to assert correctness.

Contamination, missingness, and execution integrity diagnostics. The preregistered contamination checks produced no flags: analysis/contamination_summary.csv is empty. Missingness diagnostics record complete_pairs = 120 and zero failed or unscorable episodes (analysis/missingness_summary.csv). The deterministic_reconciliation artifact confirms episode and pair accounting consistency and that all deterministic consistency checks passed.

Interpretation of the degenerate concordance. The observed perfect concordance (all pairs accepted by both downstream arms) yields a degenerate statistical picture: the paired difference estimate is point-mass zero, the bootstrap CI collapses to [0.00,0.00], and McNemar's test yields $p = 1.0$. These statistics correctly quantify uncertainty given the observed data but arise from the preregistered decision to share LLM candidates between arms. Because generator variance was removed by design, the confirmatory outcome answers the narrower question of validator-acceptance parity for identical candidates rather than the broader question of independent generator or repair superiority. This is consistent with the preregistration which specified shared_initial_candidate semantics and is explicitly recorded in execution/model_configuration.json and the execution manifest.

Why repairs were not applied in this run (artifact-grounded observation). The archived per-task traces indicate that shared LLM candidates either matched the canonical normalized reference (exact match) or satisfied the validator's normalization and dependency checks (ordering-insensitive acceptance). Representative normalized_response and normalized_reference_answer fields in execution/scoring/task-000XXX-*.json demonstrate this pattern: validator normalization reconciled minor syntactic/order variations and all required commands and final state constraints were present, so no repair loop was triggered. This operational detail explains the observed repair_applied = false counters without invoking any unarchived assumptions.

Consequences for preregistered hypotheses and next steps. H1 (paired difference in end-to-end correctness A - B) is measured here but under the restricted estimand implied by shared_initial_candidate semantics (validator-acceptance parity). H2 (agentic repair coverage advantage) is unobserved because no repair events occurred. The artifact bundle and execution accounting provide explicit levers for a discriminating follow-up (remove shared candidate to reintroduce generator variance, increase model_calls_used to produce independent per-arm generations, enable and instrument repair attempts) — each such change would produce verifiable differences in archived metrics (hosted_model_call_event_count, nonzero repair_applied flags, increased model_calls_used) that would support testing the originally preregistered repair-and-generation hypotheses.

V. Discussion

What the run measures — and what it does not. Because shared_initial_candidate semantics were used, the preregistered confirmatory estimand reduces to validator-acceptance parity: given identical LLM candidates, do downstream orchestration and verification paths differ in deterministic validator acceptance? The observed complete concordance ($n11 = 120$) answers this restricted question: both validators accepted the identical candidates for every paired task. The run does not exercise independent generator variance, sampling effects, or repair efficacy; therefore claims about which architecture would produce more correct initial candidates or achieve higher repair coverage when generating independently are untested.

Artifact-grounded reasons for concordance. The execution and scoring artifacts provide concrete evidence that explains why concordance was observed. Representative per-task scoring JSONs show that for many tasks the shared_initial_candidate matched the canonical reference answer after normalization (see execution/scoring/task-000001-baseline.json and similar files). The deterministic task generator encodes a required_sequence and an explicit canonical reference in each manifest entry (execution/task_manifest.jsonl), and the validator checks required_command_count and parsed_command_count against those expectations. In this run the normalized_response frequently fulfilled the required_command_count and ordering constraints (examples: parsed_command_count = required_command_count = 4 for pair-000001), producing validation_after.valid = true and repair_applied = false. Those per-task fields are the concrete mechanistic link between generation and validator acceptance and are archived in execution/scoring/*.json.

Design consequences and how to obtain a discriminating comparison. The shared_candidate design intentionally removes generator variance; to test the original H1/H2 claims about independent generation and repair performance the preregistration must be changed in ways that will be visible in the artifacts. Concretely: (i) remove shared_initial_candidate so each arm receives an independent LLM generation per pair — this change would increase model_calls_used from 120 to 240 for initial candidates (the preregistration documents this resource implication); (ii) enable independent sampling (multiple initial candidates per arm) to estimate generator variance rather than a single sample; (iii) enable and instrument repair iterations (record nonzero repair_applied flags and repair_provider_trace paths) so secondary estimands (repair coverage, mean repair iterations) become observable; (iv) optionally increase emulator fidelity or include hardware runs to reduce simulator/hardware divergence risk. Each such modification produces distinct, archived signatures: higher hosted_model_call_event_count, model_calls_used > 120, nonzero repair_applied counts in execution/scoring/*.json, and a nondegenerate analysis/paired_contingency_table.csv. These artifact changes directly enable the originally preregistered confirmatory tests.

Construct and measurement nuances. Deterministic validator acceptance is a conservative binary correctness signal and a practical operational metric for automated deployment guards, but it compresses several dimensions into a single pass/fail outcome. The per-task validator traces (validation_before/after, parsed_command_count, required_command_count, violation_codes) contain richer signals that can be analyzed to measure partial correctness, ordering violations, or near misses without relaxing the binary deployment gate. Using those intermediate fields would allow graded performance metrics (e.g., fraction of required commands present, counts of transient dependency violations) while preserving a strict deployment threshold for production gates. The current run preserves those richer traces in execution/scoring/*.json; future analyses can reuse them to report more nuanced construct validity metrics without changing the production-gate semantics.

Why repairs were unobserved here. The execution artifacts show repair_applied = false for all episodes. This outcome follows from two joint facts present in the archived records: the deterministic generator/task calibration that produced candidates matching canonical references in many cases (task_manifest entries include explicit required_sequence and reference_answer) and the single-sample shared generation per pair. Because the validator accepted these candidates outright, the repair subsystem was not invoked; this is a valid experimental outcome but it leaves preregistered secondary estimands (repair coverage, mean repair iterations) unobserved. Recording and archiving the absence of repairs is itself informative and is reported in analysis/condition_summary.csv and the per-task scoring JSONs.

Operational interpretation and cautions for practitioners. Validator-acceptance parity for identical candidates indicates that, when given the same configuration text, translation-first and agentic orchestration paths can reach identical deterministic verdicts under the chosen validator and emulator. It does not imply that either architecture will produce safer, more maintainable, or more explainable configurations when operating autonomously with independent generation and repair. Practitioners should therefore treat validator-acceptance parity as one necessary but not sufficient condition for architectural equivalence: additional evaluations that exercise independent generator sampling, repair loops, and human-operator judgment remain essential before claiming production interchangeability.

Threats to validity revisited in light of artifacts. Internal validity: shared_candidate removed generator variance by design and guaranteed identical upstream inputs; this is a deliberate design choice that narrows rather than threatens internal validity for the measured estimand. Construct validity: the binary validator acceptance captures a strict correctness definition but omits maintainability and explanation dimensions; however, per-task parsed counts and violation codes available in the archive enable complementary construct checks. External validity: the synthetic Mininet-derived corpus, single model (gpt-5-mini), and single deterministic validator limit generality; these constraints are explicit in the preregistration and in the limitations section. Conclusion validity: the degenerate contingency (all concordant) correctly yields a point mass CI and p = 1.0; the analysis artifacts (analysis/analysis_log.jsonl, analysis/paired_contingency_table.csv) document the exact procedures and seeds used so that alternate designs that produce nondegenerate discordance can be power-analyzed and reproducibly executed.

VI. Limitations

- Preregistered shared_initial_candidate semantics were used; this intentionally removes generator variance and prevents this run from assessing independent generation robustness differences between architectures.
- The task corpus was synthetic and executed in an emulator (Mininet-derived topologies and public vendor example grammars); emulator-to-hardware divergence limits external validity to production devices.
- A single LLM (gpt-5-mini) and a single deterministic validator (deterministic_netops_validator_v5) were used; results do not imply multi-model or multi-validator generality.
- No repairs were applied in this run (repair_applied = false in per-task traces); preregistered secondary estimands about repair coverage remain unobserved for this execution.
- The binary deterministic validator acceptance metric does not capture operator-facing properties such as explanation quality, maintainability, or human trust, which matter in production but were outside the metric used here.

VII. Conclusion

We executed a preregistered, artifactized paired experiment comparing translation-first and agentic generate→validate→repair NetOps pipelines under shared_initial_candidate semantics. For 120 paired tasks deterministic validators accepted identical LLM candidates in both arms for every pair (n11 = 120, n10 = n01 = n00 = 0), yielding paired difference 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar p = 1.0). No repairs were applied (repair_applied = false for all episodes); preregistered hypotheses about repair coverage are therefore unobserved. The run precisely measures validator-acceptance parity under controlled shared candidates and provides a reproducible artifact bundle and explicit protocol modifications for future discriminating comparisons that exercise independent generation and repair coverage.

DISCLOSURE STATEMENT

This research was executed autonomously after the autonomy lock under the frozen master prompt archived with the run provenance. Preregistration: CAND-2026-001-prereg-v1 (archived and used to freeze the design; preregistration_sha256 recorded in the execution manifest). Generation used gpt-5-mini (declared_model_version in execution/model_configuration.json); provider record 'openai_responses'. The hosted adapter family was hosted_netops_gvr_v1. Deterministic artifacts included deterministic_netops_task_generator_v5 and deterministic_netops_validator_v5 (validator_version 5.0). The run deliberately used the preregistered shared_initial_candidate generation semantics: both pipeline arms received the same initial LLM candidate per task; execution accounting shows model_calls_used = 120 (one shared generation per pair) while planned episodes were 240. Primary analysis used paired bootstrap percentile CIs (10,000 resamples, seed 1729) and an exact McNemar two-sided test executed by paired_binary_analysis_v1. No human manually edited, rewrote, or post-processed technical content after the autonomy lock. The Research Director selected the broad topic family and constrained the initial master prompt prior to the autonomy lock; the autonomous run performed literature review, final research-question selection, preregistration, code generation, experiment execution, analysis, peer review, and manuscript drafting after the lock. Immutable reference to the initial master prompt: provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). The full execution and analysis artifact bundle is archived and referenced by the execution manifest included with this submission.

REFERENCES

[1] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, David Walker, "Don't Mind the Gap", 2016, 10.1145/2934872.2934909

[2] Radu Stoenescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881

[3] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391

[4] Umar Mahmood, Wang-Cheol Song, "A Self-Correcting Multi-Agent LLM Pipeline for Verified Kubernetes Network Policy", 2026, 10.23919/ifipnetworking70592.2026.11578993

[5] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318